

SchnelPay – Phase 3.2: Withdrawal Delay + Broadcast Path (Design Doc V2)

Status: DESIGN – no code written yet. Updated after TWO independent external

security reviews. For a second design-review pass + later implementation review.

Context: Custodial withdrawals are DISABLED/inactive. PRE-CUSTODIAL gate –

must be built + tested before custodial goes live; zero live risk building now.

Custody model: via a licensed BaaS/CaaS partner, NOT DIY key custody. The

broadcast step is a partner API call, not raw signing. Delay / state machine /

idempotency / cancellation / snapshot are partner-agnostic and buildable now; the

broadcast call itself is a stub until the partner is chosen.

0. What changed in V2 (review synthesis)

Two reviewers (independent) converged strongly. The big changes from V1:

#ChangeSourceV1 saidV2 says1Fail–STOPPED on bad configBothlong default (60m)HALT + alert; never auto-broadcast on bad config2Approval snapshot + pre-broadcast verificationBoth (mandatory)(missing)snapshot the approved amount/address; re-verify before broadcast3HMAC tamper-evidence on approvalReviewer 1(missing)sign critical fields at approval; recheck at broadcast4reconciling state + reconciliation workerReviewer 2only failed/completedadd reconciling for "partner timed out, outcome unknown"5Outbox + deterministic idempotency keyReviewer 1single status fieldddedicated key =

SHA256(tx_id+addr+amount); poll-before-rebroadcast6Immutable withdrawal_events ledgerReviewer 2(missing)append-only event log for audit/forensics7User self-cancellationBoth (yes)open questionallow it, rate-limited by user ID8Insider / malicious-admin threatBothsession-focused onlyexplicit threat + layered controls9Partner idempotency = selection requirementBoth(implied)hard requirement on the custody-partner checklist10Worker: Option A confirmed for launchBothleaning AA confirmed; isolate later, not day-one

The three non-negotiable pre-custodial controls (both reviewers + internal agree):

Immutable approval snapshot + pre-broadcast verification (TOCTOU defense)

Reconciliation state + workflow ("we don't know if it sent")

Partner idempotency (deterministic keys + status-lookup API)

These three eliminate the majority of catastrophic withdrawal-failure scenarios

that hit early crypto/fintech platforms.

0a. Internal adjudication where reviewers differed or need nuance
Recorded honestly so we don't over-trust any single recommendation:

HMAC vs. snapshot (R1 vs R2): Do BOTH – they defend different things.
Snapshot

= human-readable audit + comparison. HMAC = tamper-detection even if
the row is

edited. Caveat neither fully stressed: an HMAC is only as strong as
the secret's

isolation. If the DB is compromised AND the HMAC secret sits in an env
var the same

attacker reads, they can re-sign a tampered row. So HMAC meaningfully
defends against

DB-only compromise (SQLi, rogue DB access), NOT full-host compromise.

Under the

BaaS-partner model the partner's own approval controls are the deeper
backstop.

-> Treat HMAC as defense-in-depth, not a silver bullet. [REVIEW]
secret-isolation

options at our scale (separate KMS vs. separate env on a different
surface)?

worker_threads/fork isolation (R1): Good instinct, but adds real
complexity

(IPC, separate DB pool, error propagation). At launch volume, a plain
in-process

interval with strict per-call await + timeout won't meaningfully block
the loop.

-> Launch with a plain interval + hard timeouts; move to
worker_threads/separate

service at volume. (R2 agrees: don't over-build now.)

Outbox + full reconciliation loop (R1): Correct for maturity, but a
meaningful

build, and the exact idempotency-key format + partner status-lookup
API shape it.

-> Build the seams now (events ledger + reconciling state + snapshot
get most of

the value); implement the full outbox + poll-before-rebroadcast loop
at partner

integration, when the key format and lookup API are known.

1. Current state – what EXISTS today (verified from source)

1.1 Withdrawal lifecycle today [EXISTS]

User requests -> CustodialService.requestWithdrawal() -> INSERT
status='pending'

Admin approves -> approveWithdrawal() -> 'pending' -> 'processing'
[PIN-gated, Phase 2.1 done]

Admin rejects -> rejectWithdrawal() -> 'pending' -> 'cancelled'
[PIN-gated, Phase 2.1 done]

... then NOTHING. Never broadcasts. <-- THE GAP

1.2 State machine the schema anticipates [EXISTS – schema]
custodial_transactions.status CHECK:
pending -> detected -> confirming -> processing -> broadcasting ->
completed

-> failed ->

cancelled

Supporting columns already present: tx_hash, block_number,
confirmations/
required_confirmations (default 6), withdrawal_address,
withdrawal_approved_at,
withdrawal_approved_by, screening_status, completed_at, metadata
JSONB.

1.3 Confirmed MISSING [EXISTS – verified absent]

No broadcast execution (nothing past 'processing').

No WITHDRAWAL_DELAY (code/env/schema).

No delay/schedule column.

No background-worker infra (only in-process setTimeout in
payment.service.ts).

Payment "broadcast" is MOCKED (generateMockTxHash(), setTimeout(5000)
"confirm").

Custodial addresses are demo-generated (generateDemoAddress(),
"replace with real

KMS" comment) – confirms no real keys yet (consistent with BaaS-
partner plan).

1.4 Stack facts (for the reviewer mapping lock syntax) [EXISTS]

PostgreSQL (Railway), pg Pool. Note: production Postgres runs
autocommit-style
across separate -c calls – real transactions need a single psql/client
session.

Express + TypeScript, single Railway instance.

Redis available (used for rate-limiting + PIN lockout) – usable for
cancel

rate-limiting and worker coordination.

2. Phase 3.2 goals (updated)

Configurable delay (WITHDRAWAL_DELAY_MINUTES) between approval and
broadcast.

Fail-STOPPED on missing/invalid config (halt + alert; never auto-
broadcast).

Approval snapshot + HMAC, verified immediately before broadcast
(TOCTOU defense).

Cancellation window: admin (PIN-gated) AND user self-cancel (rate-
limited).

Idempotent broadcast worker: at-most-once broadcast across crash/
restart/concurrency.

reconciling state + workflow for unknown-outcome partner calls.
Immutable withdrawal_events ledger for audit/forensics.
Safe, guarded state transitions throughout.

Layers above Phase 2.1 (PIN approval done) and below Phase 3.1 (TOTP, later).

3. Proposed design (V2)

3.1 Schema additions [PLANNED]

```
sqlALTER TABLE custodial_transactions
  ADD COLUMN broadcast_after      TIMESTAMP,          -- broadcast-
eligible time (frozen at approval)
  ADD COLUMN approval_snapshot   JSONB,              -- {amount, asset,
destination_address, approved_by, approved_at}
  ADD COLUMN approval_signature  VARCHAR(128);       --
HMAC_SHA256(tx_id+amount+address, SECRET)

-- expand status CHECK to add 'reconciling'
-- pending, detected, confirming, processing, broadcasting,
reconciling, completed, failed, cancelled

CREATE INDEX idx_withdrawals_broadcast_after
  ON custodial_transactions(status, broadcast_after); -- R2's
worker-query index
```

```
CREATE TABLE withdrawal_events ( -- immutable append-only ledger
(R2)
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  custodial_transaction_id UUID NOT NULL REFERENCES
custodial_transactions(id),
  event_type VARCHAR(40) NOT NULL, -- created|approved|
delay_started|claimed|broadcast_started|broadcast_completed|
reconciling|completed|failed|cancelled
  actor VARCHAR(200),             -- admin id / 'user' / 'worker' /
'system'
  detail JSONB,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
-- No UPDATE/DELETE in app code; append-only by convention (+ later:
DB-level enforcement).
[REVIEW] snapshot in a JSONB column on the row vs. a separate
immutable table?
(On-row is simpler; separate table is harder to tamper alongside the
row.)
3.2 Fail-STOPPED delay resolution [PLANNED - changed from V1]
tsfunction resolveDelayMinutes(): number {
  const raw = process.env.WITHDRAWAL_DELAY_MINUTES; // or feature flag
  if (!raw || isNaN(Number(raw))) {
    logger.emergency("CRITICAL: withdrawal delay config missing/
corrupt. Halting worker.");
```

```

    throw new ConfigurationError("System unsafe for processing
withdrawals.");
}
    return Math.max(10, Math.min(1440, Number(row))); // strict
invariants 10m..24h
}
Worker loop catches ConfigurationError -> stops claiming rows + raises
an alert.
No withdrawal auto-broadcasts while config is bad. Funds delayed =
recoverable;
funds misdirected = maybe not.
[REVIEW] confirm halt-the-loop (not crash-the-web-process) is the
right blast radius.
3.3 Approval: snapshot + sign + set delay [PLANNED]
Modify the Phase 2.1 approveWithdrawal (inside the same PIN-gated
handler):
snapshot = { amount, asset, destination_address, approved_by,
approved_at }
signature = HMAC_SHA256(tx_id + amount + destination_address,
APPROVAL_SECRET)
broadcast_after = NOW() + resolveDelayMinutes()
status = 'processing'
+ INSERT withdrawal_events('approved' / 'delay_started')
3.4 Pre-broadcast verification [PLANNED – the TOCTOU defense]
When the worker claims a row, BEFORE any partner call:
recompute signature from CURRENT row fields
if signature != stored approval_signature -> LOCK DOWN (frozen/
flagged) + alert
compare CURRENT amount/address to approval_snapshot -> any mismatch ->
LOCK DOWN + alert
Defends against address-substitution / amount-modification during the
delay window
(SQLi, rogue DB edit, insider, bug). [REVIEW] lock to failed, or a new
frozen/flagged state requiring manual review? (Probably the latter –
don't let an
automated path resolve a tamper event.)
3.5 Idempotent broadcast worker [PLANNED – Option A confirmed]
Launch architecture: in-process interval in the Express backend, await
with a hard
per-call timeout (no worker_threads yet). Claim pattern (optimistic
lock):
sqlUPDATE custodial_transactions
    SET status='broadcasting', updated_at=NOW()
    WHERE id=$1 AND status='processing' AND broadcast_after <= NOW();
-- rowCount=1 -> we own it; rowCount=0 -> taken/ineligible
Batch claim with SELECT ... FOR UPDATE SKIP LOCKED. Deterministic
idempotency key
for the (future) partner call: SHA256(tx_id + withdrawal_address +
amount) – never
random. Full outbox + poll-before-rebroadcast reconciliation loop =

```

built at partner
integration (needs the partner's status-lookup API). Broadcast itself
= stub now.
[REVIEW] at launch volume, is a single deterministic key + reconciling
state
enough, or is the separate outbox table worth building before the
partner?
3.6 reconciling state [PLANNED – R2]
If a (future real) partner call times out / returns ambiguously ->
status=reconciling,
NOT failed. A reconciliation step (built at partner integration) polls
the partner by
the deterministic key to learn the true outcome before any retry. For
now the stub
never produces this, but the state + transitions exist and are tested.
3.7 Cancellation [PLANNED]

Admin: PIN-gated endpoint (reuse Phase 2.1 checkAdminPin), WHERE
status='processing'.
User self-cancel: allowed while status='processing' AND
broadcast_after > NOW(),
rate-limited by user ID (Redis) to block griefing. Turns users into
distributed
fraud sensors (notify on approval -> "cancel" button).
Race vs. worker: the atomic conditional UPDATE (WHERE
status='processing') makes
whichever fires first the single winner; loser gets rowCount=0. Append
a
cancelled / claimed event either way.

4. Threat model (V2 – expanded)

ThreatMitigationCompromised admin session approves frauddelay window +
cancellation; PIN (2.1); TOTP (3.1)Malicious administrator (legit PIN
holder)delay + TOTP + immutable event ledger + snapshot verify (no
instant/unlogged money move)Address-substitution during delay
(TOCTOU)approval snapshot + HMAC re-verify before broadcastAmount-
modification during delay (TOCTOU)approval snapshot + HMAC re-verify
before broadcastDouble-broadcast / user paid twiceoptimistic-lock
claim + deterministic idempotency key + partner idempotencyPartner
replay (processes twice)mandatory partner-side idempotency
key"Broadcast succeeded, DB write failed"reconciling state + poll-
before-rebroadcast (at partner integration)Misconfig silently disables
protectionfail-STOPPED (halt + alert)Worker crash mid-flightidempotent
claim + reconciling + reconciliationProcessed from wrong
stateconditional-UPDATE state guardsCancel-vs-broadcast raceatomic
single-winner UPDATEUser self-cancel griefingRedis rate-limit by user
ID

5. Custody-partner selection requirements (NEW – both reviewers)

Make these HARD requirements when choosing the BaaS/CaaS partner:

Idempotent withdrawal API (Idempotency-Key / client request ID on creation).

Withdrawal status-lookup API (so reconciliation can ask "did this go through?").

Request correlation IDs; partner-side audit logs; webhook retry support.

Without idempotency + status-lookup, crash recovery is dangerous. Add to the partner checklist now.

6. Platform security/operational profile (NEW – to state explicitly, per R2)

Security controls: withdrawal delay window; PIN-gated approval (done); TOTP approval

(future 3.1); immutable approval snapshot; pre-broadcast verification; partner

idempotency keys; reconciliation workflow; user cancellation in delay window;

immutable event ledger; fail-stopped on invalid config.

Operational controls: broadcast worker; reconciliation worker (at scale); documented

recovery procedures; audit reporting (ties to the existing admin audit-panel workstream).

7. Out of scope for 3.2 (keep review focused)

Phase 3.1 TOTP; custody-partner choice; real KMS/keys (partner's); sanctions/address

screening (separate, screening_status exists); post-broadcast confirmation tracking.

8. Build order (V2)

Migration: broadcast_after, approval_snapshot, approval_signature, expand status

CHECK (reconciling), withdrawal_events table, index.

resolveDelayMinutes() – fail-stopped (+ tests).

Approval: snapshot + HMAC sign + set broadcast_after + emit events (modify 2.1 handler).

Pre-broadcast verification (HMAC + snapshot recheck → lockdown on mismatch).

Broadcast worker (Option A, in-process interval, hard timeouts) with conditional-claim

idempotency + stubbed partner broadcast + event emission.

Cancellation: PIN-gated admin + rate-limited user self-cancel.

Tests: delay enforced; fail-stopped on bad config; no double-broadcast under

concurrency; cancel-vs-broadcast race; restart safety; tamper →

lockdown.

(At partner selection) real broadcast adapter + deterministic idempotency key + full reconciliation poll-before-rebroadcast + outbox if warranted.

9. Open questions for the next review pass

Section 3.1 snapshot on-row JSONB vs. separate immutable table?

Section 3.2 halt-the-worker vs. crash-the-process blast radius?

Section 3.4 tamper-detected -> failed vs. a manual-review frozen state?

Section 3.5 deterministic key + reconciling enough at launch, or build the outbox now?

Section 0a HMAC secret isolation at our scale (the "secret in same DB" limitation)?

Event-ledger immutability – convention now, DB-enforced (triggers/permissions) when?

Design V2, post-review. Implementation review to follow on actual code. Nothing live; custodial withdrawals remain disabled.